

Logistic Regression and Text Classification [4]

Thành viên thực hiện:

Hà Thanh Phong - 24520024

Hà Xuân Thiện - 24520031

Trần Quang Trường - 24521901

University of Information Technology - VNUHCM

Ngày 3 tháng 5 năm 2026

- 1 Learning in Multinomial Logistic Regression
- 2 Evaluation: Precision, Recall, F-measure
- 3 Test set and cross-validation

- 1 Learning in Multinomial Logistic Regression
- 2 Evaluation: Precision, Recall, F-measure
- 3 Test set and cross-validation

Nhắc lại công thức hàm $L_{\text{CE}}(\hat{y}, y)$ cho binary logistic regression:

$$L_{\text{CE}}(\hat{y}, y) = -\log p(y \mid x) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})] \quad (4.37)$$

Công thức tính hàm $L_{\text{CE}}(\hat{\mathbf{y}}, \mathbf{y})$ cho Multinomial Logistic Regression:

$$L_{\text{CE}}(\hat{\mathbf{y}}, \mathbf{y}) = - \sum_{k=1}^K y_k \log \hat{y}_k \quad (4.38)$$

$$= -\log \hat{y}_c, \quad (\text{Với } c \text{ là correct class}) \quad (4.39)$$

$$= -\log \hat{p}(y_c = 1 \mid \mathbf{x})$$

$$= -\log \frac{\exp(\mathbf{w}_c \cdot \mathbf{x} + b_c)}{\sum_{j=1}^K \exp(\mathbf{w}_j \cdot \mathbf{x} + b_j)} \quad (4.40)$$

Công thức tính đạo hàm riêng theo $\partial w_{k,i}$:

$$\begin{aligned}\frac{\partial L_{\text{CE}}}{\partial w_{k,i}} &= -(y_k - \hat{y}_k) x_i \\ &= -(y_k - p(y_k = 1 \mid x)) x_i \\ &= -\left(y_k - \frac{\exp(\mathbf{w}_k \cdot \mathbf{x} + b_k)}{\sum_{j=1}^K \exp(\mathbf{w}_j \cdot \mathbf{x} + b_j)} \right) x_i\end{aligned}\tag{4.41}$$

- 1 Learning in Multinomial Logistic Regression
- 2 Evaluation: Precision, Recall, F-measure
- 3 Test set and cross-validation

Evaluation: Precision, Recall, F-measure

Bắt đầu với bài toán **binary detection**, mục tiêu của bài toán thường phân loại data vào một trong hai class (positive hoặc negative).

Ví dụ với bài toán kiểm tra tin nhắn spam (spam detection):

- Mục tiêu của bài toán là xác định các tin nhắn được coi là **spam** (positive) hay **không spam** (negative).

Evaluation: Precision, Recall, F-measure

Bắt đầu với bài toán **binary detection**, mục tiêu của bài toán thường phân loại data vào một trong hai class (positive hoặc negative).

Ví dụ với bài toán kiểm tra tin nhắn spam (spam detection):

- Mục tiêu của bài toán là xác định các tin nhắn được coi là **spam** (positive) hay **không spam** (negative).
- Tuy nhiên, để biết biết một tin nhắn có **thật sự** là spam hay không thì vẫn phải cần con người nhận định.
- Những label (spam hoặc không spam) mà do con người nhận định còn được gọi là **gold labels**.

Để đánh giá các hệ thống cho các bài toán detection như trên, công cụ cơ bản nhất hay sử dụng là **confusion matrix** (ma trận nhầm lẫn).

Confusion matrix là một ma trận 2 chiều gồm một chiều là **gold labels** (Nhãn đúng), và một chiều là **system output labels** (Nhãn dự đoán của hệ thống).

Evaluation: Precision, Recall, F-measure

Xét ví dụ về **confusion matrix** cho bài toán phân loại nhị phân (binary classification):

		<i>gold standard labels</i>		
		gold positive	gold negative	
<i>system output labels</i>	system positive	true positive	false positive	$\text{precision} = \frac{tp}{tp+fp}$
	system negative	false negative	true negative	
		$\text{recall} = \frac{tp}{tp+fn}$		$\text{accuracy} = \frac{tp+tn}{tp+fp+tn+fn}$

Evaluation: Precision, Recall, F-measure

Một số định nghĩa:

- **True positive (TP)**: system đoán positive, **và nó đúng** là positive.
- **False positive (FP)**: system đoán positive, **nhưng nó lại** là negative.
- **False negative (FN)**: system đoán negative, **nhưng nó lại** là positive.
- **True negative (TN)**: system đoán negative, **và nó đúng** là negative.

Accuracy là một thước đo cho biết **tỷ lệ dự đoán đúng** trên **toàn bộ mẫu dữ liệu**. Công thức của Accuracy:

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}$$

Evaluation: Precision, Recall, F-measure

Accuracy là một thước đo cho biết **tỷ lệ dự đoán đúng trên toàn bộ mẫu dữ liệu**. Công thức của Accuracy:

$$\mathbf{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}$$

Tuy nhiên, **Accuracy** không phải lúc nào cũng phản ánh đúng hiệu quả của mô hình.

Giả sử dữ liệu có 1,000,000 trường hợp negative, nhưng chỉ có 10 trường hợp positive. Nếu system luôn đoán là negative thì accuracy vẫn gần như 100%.

Precision là một thước đo cho biết tỉ lệ mẫu **thật sự là positive** trên **toàn bộ mẫu** mà hệ thống **đ đoán là positive**:

$$\text{Precision} = \frac{\text{true positives}}{\text{true positives} + \text{false positives}}$$

Evaluation: Precision, Recall, F-measure

Precision là một thước đo cho biết tỉ lệ mẫu **thật sự là positive** trên **toàn bộ mẫu** mà hệ thống **đ đoán là positive**:

$$\text{Precision} = \frac{\text{true positives}}{\text{true positives} + \text{false positives}}$$

Recall là thước đo cho biết tỉ lệ mẫu mà hệ thống **đ đoán đúng** trên tất cả các mẫu **thật sự là positive**:

$$\text{Recall} = \frac{\text{true positives}}{\text{true positives} + \text{false negatives}}$$

Evaluation: Precision, Recall, F-measure

Bên cạnh Precision và Recall, có nhiều độ đo kết hợp từ hai độ đo trên. Đơn giản nhất là **F-measure**:

$$F_{\beta} = \frac{(\beta^2 + 1)PR}{\beta^2 P + R}$$

Hệ số β được dùng để điều chỉnh mức độ đóng góp của precision và recall trong F-measure:

Evaluation: Precision, Recall, F-measure

Bên cạnh Precision và Recall, có nhiều độ đo kết hợp từ hai độ đo trên. Đơn giản nhất là **F-measure**:

$$F_{\beta} = \frac{(\beta^2 + 1)PR}{\beta^2 P + R}$$

Hệ số β được dùng để điều chỉnh mức độ đóng góp của precision và recall trong F-measure:

- Nếu $\beta > 1$, recall sẽ được coi trọng hơn precision.
- Nếu $\beta < 1$, precision sẽ được ưu tiên hơn recall.
- Khi $\beta = 1$, precision và recall được xem trọng như nhau, nên đây là **trường hợp cân bằng**.

Evaluation: Precision, Recall, F-measure

Bên cạnh Precision và Recall, có nhiều độ đo kết hợp từ hai độ đo trên. Đơn giản nhất là **F-measure**:

$$F_{\beta} = \frac{(\beta^2 + 1)PR}{\beta^2 P + R}$$

Trường hợp cân bằng ($\beta = 1$) là trường hợp được sử dụng rộng rãi nhất, còn được gọi là $F_{\beta=1}$ hay F_1 :

$$F_1 = \frac{2PR}{P + R} \quad (4.42)$$

Evaluation: Precision, Recall, F-measure

F-measure được tính bằng **weighted harmonic mean** (trung bình điều hòa có trọng số) giữa precision và recall.

Harmonic mean (trung bình điều hòa) là nghịch đảo của trung bình cộng các số nghịch đảo:

$$\text{HarmonicMean}(a_1, a_2, a_3, a_4, \dots, a_n) = \frac{n}{\frac{1}{a_1} + \frac{1}{a_2} + \frac{1}{a_3} + \dots + \frac{1}{a_n}} \quad (4.43)$$

Evaluation: Precision, Recall, F-measure

F-measure được tính bằng **weighted harmonic mean** (trung bình điều hòa có trọng số) giữa precision và recall.

Harmonic mean (trung bình điều hòa) là nghịch đảo của trung bình cộng các số nghịch đảo:

$$\text{HarmonicMean}(a_1, a_2, a_3, a_4, \dots, a_n) = \frac{n}{\frac{1}{a_1} + \frac{1}{a_2} + \frac{1}{a_3} + \dots + \frac{1}{a_n}} \quad (4.43)$$

Và do đó, F-measure được viết thành:

$$F = \frac{1}{\alpha \frac{1}{P} + (1 - \alpha) \frac{1}{R}} \quad \text{hay} \quad F = \frac{(\beta^2 + 1)PR}{\beta^2 P + R} \quad (4.44)$$

với $\beta^2 = \frac{1 - \alpha}{\alpha}$

Evaluation: Precision, Recall, F-measure

Những ví dụ và độ đo trên được dùng cho các bài toán có 2 class.

Vậy với các bài toán nhiều hơn 2 class thì sao?

Ví dụ:

- Sentiment Analysis (Phân tích cảm xúc) gồm 3 class (Positive, Negative, Neutral)
- Text classification (Phân loại từ)
- ...

Evaluation: Precision, Recall, F-measure

Xét ví dụ bài toán phân loại thành 3 class urgent, normal và spam:

		<i>gold labels</i>			
		urgent	normal	spam	
<i>system output</i>	urgent	8	10	1	precision_u = $\frac{8}{8+10+1}$
	normal	5	60	50	precision_n = $\frac{60}{5+60+50}$
	spam	3	30	200	precision_s = $\frac{200}{3+30+200}$
		recall_u = $\frac{8}{8+5+3}$	recall_n = $\frac{60}{10+60+30}$	recall_s = $\frac{200}{1+50+200}$	

Evaluation: Precision, Recall, F-measure

Ta có thể tính precision và recall riêng cho từng class. Nói cách khác, mỗi class sẽ được xem như một bài toán riêng để đo lường hệ thống trên chính class đó.

Class 1: Urgent			Class 2: Normal			Class 3: Spam		
	true urgent	true not		true normal	true not		true spam	true not
system urgent	8	11	system normal	60	55	system spam	200	33
system not	8	340	system not	40	212	system not	51	83

Tuy nhiên, nếu chỉ nhìn từng class riêng lẻ thì sẽ khó có được một **kết luận chung** cho toàn bộ hệ thống. Vì vậy, ta cần gộp các giá trị này lại thành một metric tổng quát hơn.

Chúng ta có thể gộp các giá trị này theo 2 cách:

- Với **Macroaveraging**, ta tính metric riêng cho từng class xong lấy trung bình cộng lại.
- Với **Microaveraging**, ta gộp nhóm các quyết định theo từng class thành một confusion matrix riêng, rồi tính precision và recall chung cho toàn hệ thống.

- 1 Learning in Multinomial Logistic Regression
- 2 Evaluation: Precision, Recall, F-measure
- 3 Test set and cross-validation

Test set and cross-validation

Trong quá trình train và test cho text classification, ta sẽ có 3 tập:

- **Training set** là tập dùng để huấn luyện mô hình.
- **Development test set hay devset** là tập dùng để điều chỉnh tham số (tune parameters)
- **Test set** là tập dùng để đánh giá khả năng tổng quát hóa của mô hình ở bước cuối cùng.

Test set and cross-validation

- Trong quá trình sử dụng tập devtest, để tránh việc **quá khớp** (overfit) với tập test set, chúng ta thường hay **cố định** các tập train, dev, test.
- Điều này dẫn tới câu chuyện nếu muốn có nhiều dữ liệu để train thì tập dev hoặc test sẽ **không đủ lớn** để làm tập đại diện.
- Câu hỏi đặt ra là liệu có cách nào để sử dụng toàn bộ dữ liệu để huấn luyện (train) mà vẫn muốn sử dụng tất cả để kiểm tra (test) hay không?

Test set and cross-validation

- Trong quá trình sử dụng tập devtest, để tránh việc **quá khớp** (overfit) với tập test set, chúng ta thường hay **cố định** các tập train, dev, test.
- Điều này dẫn tới câu chuyện nếu muốn có nhiều dữ liệu để train thì tập dev hoặc test sẽ **không đủ lớn** để làm tập đại diện.
- Câu hỏi đặt ra là liệu có cách nào để sử dụng toàn bộ dữ liệu để huấn luyện (train) mà vẫn muốn sử dụng tất cả để kiểm tra (test) hay không? $\Rightarrow$ Sử dụng **cross-validation**.

Test set and cross-validation

- Với một giá trị k ta chọn trước, ta chia tập dữ liệu thành k một tập con gọi là **folds**.
- Bây giờ, khi ta chọn một trong k tập folds làm tập test set thì ta có thể huấn luyện trên $k - 1$ tập còn lại và tính **error rate** (tỉ lệ lỗi) trên tập test set.
- Lặp lại với $k - 1$ tập folds còn lại, ta sẽ tính **average error rate** (tỉ lệ lỗi trung bình) thông qua trung bình error rate trong k lần chạy.

Test set and cross-validation

- Tuy nhiên, cross-validation cũng có một hạn chế là toàn bộ dữ liệu đều sẽ có lúc được dùng để test, nên ta không thể thoải mái nhìn vào dữ liệu để phân tích feature như trong nhiều bài toán NLP khác.
- Trên thực tế, người ta thường giữ một test set cố định và chưa đụng tới, còn dùng cross-validation bên trong training set để điều chỉnh tham số và chọn mô hình.

Test set and cross-validation

